

Connected Vehicle Intelligence Hackathon

ValeoSense

Solution Document

Synthetic telemetry • live fleet state • historical intelligence

Problem space chosen: Fleet operations, unsafe-driving incident detection and idling analysis.

To be submitted by: TEAM ACTION — enter your official team name.

Team members & roles: TEAM ACTION — list each member's name, role and email address.

Repository URL: <https://github.com/Kaileshwar16/VeloSense>

Repository URL comes from the local origin remote; reviewer access and latest remote contents have not been verified.

Demo video URL (≤ 5 minutes): TEAM ACTION — add an accessible recording link.

Date of submission: TEAM ACTION — enter the actual submission date (DD/MM/YYYY).

Document prepared: 01/10/2026. Source revision: c5549e8.

Submission format: PDF exported from this editable Word document.

Evidence boundary

Technical sections use repository source, saved artifacts and verification records dated 27–30 September 2026, cross-checked against the supplied 10-page Motorq Hackathon Problem Statement. This document preparation did not rerun the application or benchmarks. “Done” means implemented with stated local evidence; it does not mean production qualified. The PDF’s full production-quality bar is not met. Yellow TEAM ACTION text identifies items the submitting team must finalize.

Readiness: technical narrative completed; team details, final video/timestamps, final repository/tag checks and unresolved evidence gaps remain.

Table of Contents

1. Executive Summary
2. Problem Statement & Validation
3. Solution Description
4. Feature List
5. Solution Architecture (High-Level Design)
6. Low-Level Design
7. Non-Functional Requirements & Performance Benchmarks
8. Security & Compliance
9. Test Strategy
10. Observability
11. AI / ML Component (if used)
12. Architecture Decisions, Risks & Future Enhancements
13. Demo Video (5 Minutes Maximum)
14. Repository Checklist
15. Conclusion
16. Declarations
17. Appendix (if any)

Subsections 2.1–2.3, 3.1–3.3, 5.1–5.4 and 6.1–6.5 follow the supplied template. Supporting commands and evidence paths are included in section 17.

1. Executive Summary

ValeoSense helps fleet managers inspect current vehicle conditions, detect operational incidents and investigate recent history while telemetry continues to arrive. The prototype separates Redis live state, PostgreSQL metadata and ClickHouse analytical history behind a FastAPI service and React dashboard. A deterministic registry contains 100,000 synthetic vehicles; the normal demonstration activates a 1,000-vehicle subset.

Stateful JSON telemetry flows through Kafka-compatible Redpanda to a bounded Python processor. Detectors identify prolonged idling, speeding, harsh braking/acceleration, diagnostic faults and low energy. Existing external QueryFlux software routes history queries to ClickHouse and small recent-sample queries to embedded DuckDB. Optional Prometheus/Grafana monitoring and an independent Iceberg raw archive support inspection and replay-oriented investigation.

The saved 15-second benchmark generated 999.87 readings/sec at the stable 1K target, with 923.99 consumed records/sec including startup and drain. A 10K target was unstable. One measured ClickHouse optimization reduced median execution from 23.693 ms to 2.402 ms and rows read from 515,271 to 4,096. The 30 September verification record reports 106 Python tests passed and three opt-in integration tests skipped, plus separate real archive integration checks. No sustained 100K events/sec, customer savings or availability SLA is claimed.

2. Problem Statement & Validation

2.1 Problem Statement

A fleet manager needs a way to detect current incidents and inspect vehicle history because continuously arriving telemetry and historical scans have different access needs. Unnoticed idling, unsafe manoeuvres and diagnostic faults can delay operational intervention; the real financial and safety cost has not been measured in this project.

Primary user: fleet manager. Secondary stakeholders: dispatch/operations staff, maintenance teams and drivers. The prototype tests engineering feasibility using synthetic telemetry; it does not validate customer demand through interviews or field deployment.

2.2 Evidence & Validation

Evidence / assumption	Source or method	What it shows	Confidence
100,000 unique vehicles and VINs	artifacts/registry-verification.json; seeded CSV generator	Verified registry size; active subset is separately configured.	High for synthetic registry
Duplicate and sink replay handling	tests/integration/test_stack.py; docs/verification.md	Tested Kafka/API path and deduplicated analytical reads.	High for tested paths
Capacity boundary	benchmark_results.json; two 15-second stages	1K generation target stable; 10K unstable.	Medium; short duration
Two analytical engines execute	artifacts/dual-engine-lineage.json	Three seeded IDs found via both routes; native counters advanced.	High for saved run
Idling fuel cost	Settings assumption: 0.8 L/h × INR 100/L for ICE	INR 80 per engine-on idle hour; EV fuel cost excluded.	Assumption only

Evidence / assumption	Source or method	What it shows	Confidence
Business benefit / customer pain	No field study, interviews or customer dataset	No measured savings, incident reduction or comparative advantage.	Unvalidated

Alternatives include OEM portals, aftermarket telematics devices and integrated fleet tools such as Motorq Fuse. They were not trialled or benchmarked here. The demonstrated design choice is workload separation and traceable engine execution; no unsupported claim that existing products lack these capabilities is made.

2.3 Impact & Success Metrics

Metric	Baseline / observed	Target	Measurement
Registry correctness	100,000 rows; 100,000 unique IDs/VINS	Exactly 100,000 deterministic records	Saved registry validation
Processing capacity	Stable short 1K generation target	Template: 100K+ events/sec	Real broker + processor + both sinks; target unmet
Vehicle history lookup	23.693 ms median before	Reduce scan work	2.402 ms median after; five runs each
Dashboard / alert latency	3-second polling; no percentiles	<2 s dashboard; <5 s critical alert	End-to-end latency study pending
Operational impact	No customer baseline	Measure actionable incidents and avoidable idle time in a pilot	Simulation demonstrates mechanics only

Scale of impact: at one event per active vehicle per second, 10K active vehicles require 10K events/sec and 100K active vehicles require 100K/sec; neither is demonstrated as stable here. A 100K-row registry is not equivalent to 100K concurrently emitting vehicles. Potential benefits are faster investigation and visibility into idle time; avoided fuel, emissions, crashes or breakdowns have not been measured.

3. Solution Description

3.1 Solution Overview & User Journey

1. Start the configured Compose stack and authenticate on the dashboard.
2. Inspect fleet count, advancing vehicle timestamps and live incidents.
3. Open a vehicle to read its current state and ClickHouse history.
4. Inspect idling estimates and aggregate activity.
5. Run a bounded recent-sample query through DuckDB and inspect engine evidence.
6. Use incident context to decide on an operational follow-up; the system does not control vehicles or send driver commands.

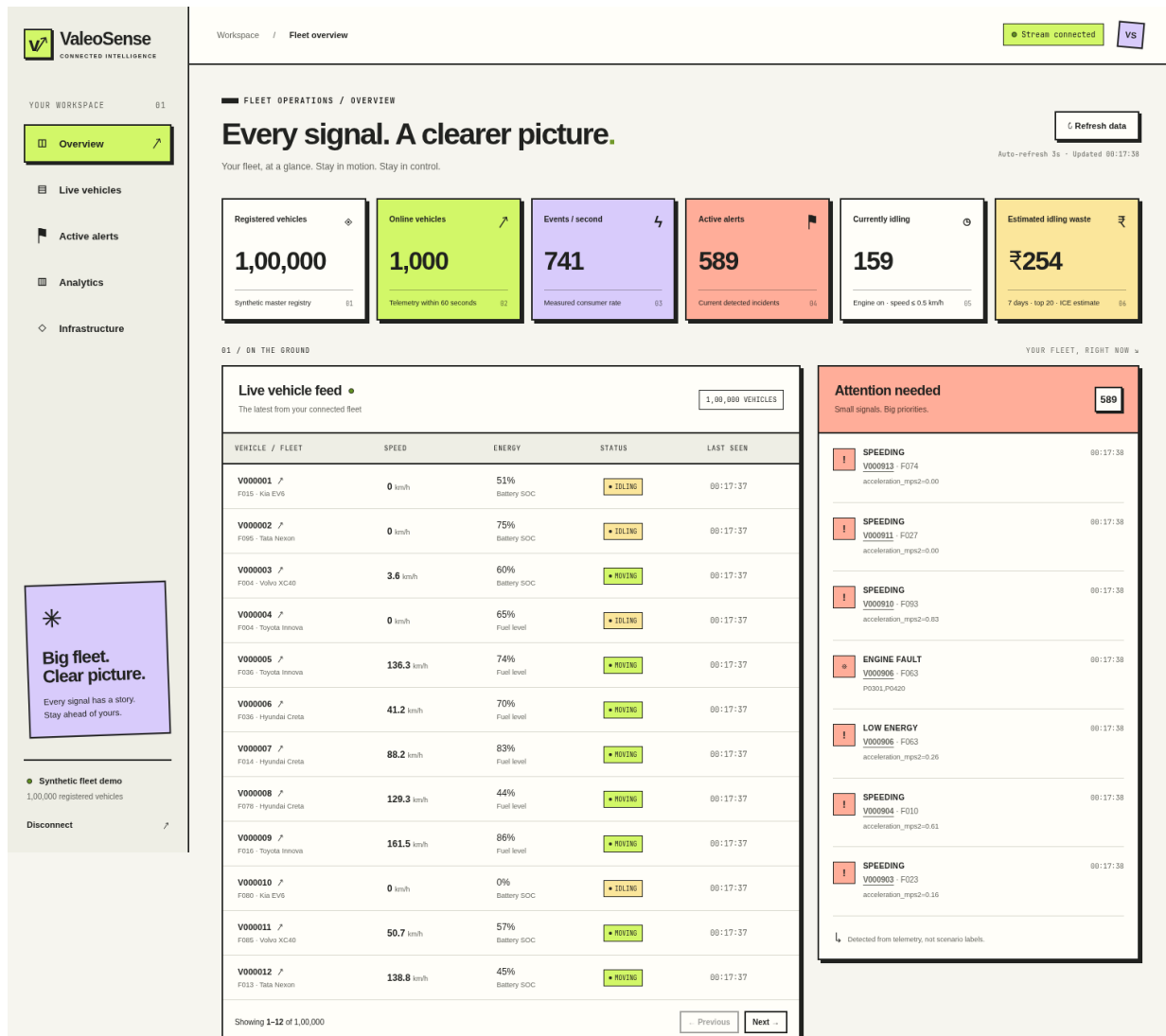


Figure 1. Working product screenshot from saved Chromium verification (September 2026), top excerpt. Displayed counts and rates are point-in-time observations, not benchmark results.

3.2 Key Value Proposition

Customer job: understand which vehicles need attention and retrieve relevant history. Pain relieved: live reads use Redis independently of analytical admission, while relational metadata and history have dedicated stores. Gain: the operator sees live state, deterministic incident context, historical records and bounded sample scope in one interface. Differentiation demonstrated by this prototype is transparent data lineage and explicit route evidence; competitive superiority has not been validated.

3.3 Innovative Ideas

Technique	Implementation and proof	Scope
Separate live and analytical access	Redis live reads; ClickHouse history; independent UI polling; analytical semaphore.	Established architecture applied to the demo.
Traceable two-engine analytics	ClickHouse → bounded Parquet → DuckDB; seeded IDs and native engine counters in dual-engine-lineage.json.	Explicit namespace routing, not automatic cost optimization.

Technique	Implementation and proof	Scope
Recoverable raw archive	Optional Iceberg data + next-offset checkpoint in one catalog transaction; raw malformed records retained.	Local archive tests; no S3 qualification or ClickHouse backfill.

4. Feature List

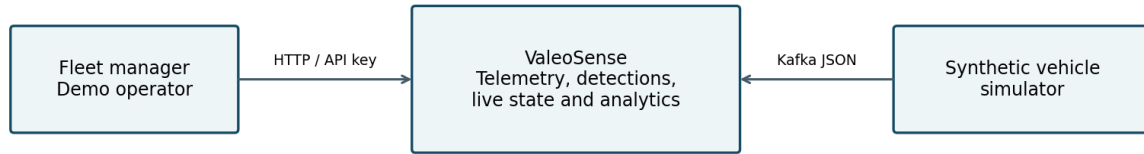
MoSCoW status: Done indicates implementation plus the stated saved local checks. Planned items are not provided by the prototype. **TEAM ACTION** — replace every “Pending” video cell for Done features with an actual timestamp from the final recording; the template asks for video evidence for each Done feature.

ID / Feature	User story	Priority	Status	Code path	Video
F-01 Synthetic fleet + 12 scenarios	As an evaluator, reproduce realistic stateful inputs.	Must	Done	simulator/; scripts/generate_vehicles.py	Pending
F-02 Incident detectors	As an operator, spot idling, unsafe manoeuvres, faults and low energy.	Must	Done	processor/detectors.py	Pending
F-03 Dedup + late-event handling	As an operator, avoid repeated incidents and stale live state.	Must	Done	processor/pipeline.py; consumer.py	Pending
F-04 Live / metadata / history APIs	As a manager, retrieve bounded authenticated fleet data.	Must	Done	backend/main.py; shared/storage.py	Pending
F-05 Dashboard + history	As a manager, inspect incidents and vehicle context.	Must	Done	frontend/src/main.jsx; api.js	Pending
F-06 QueryFlux two-engine route	As an analyst, query history or a disclosed recent sample.	Should	Done	backend/analytics.py; backend/queryflux/; scripts/duckdb_snapshot.py	Pending
F-07 Prometheus / Grafana	As an operator, inspect lag, throughput and engine metrics.	Could	Done	backend/monitoring.py; infra/monitoring/	Pending
F-08 Iceberg raw archive	As an operator, preserve raw records and read snapshots.	Could	Done	archive/; tests/unit/test_iceberg.py	Pending
F-09 Multi-tenant identity / HA	As a production owner, isolate fleets and survive node loss.	Won't in prototype	Planned	No implementation	N/A

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

System context · implemented local prototype



No OEM cloud, identity provider, live vehicle feed or runtime LLM is connected.

Figure 2. C4-style context: one local system, one operator and synthetic vehicle input.

Containers · write path and independent read paths

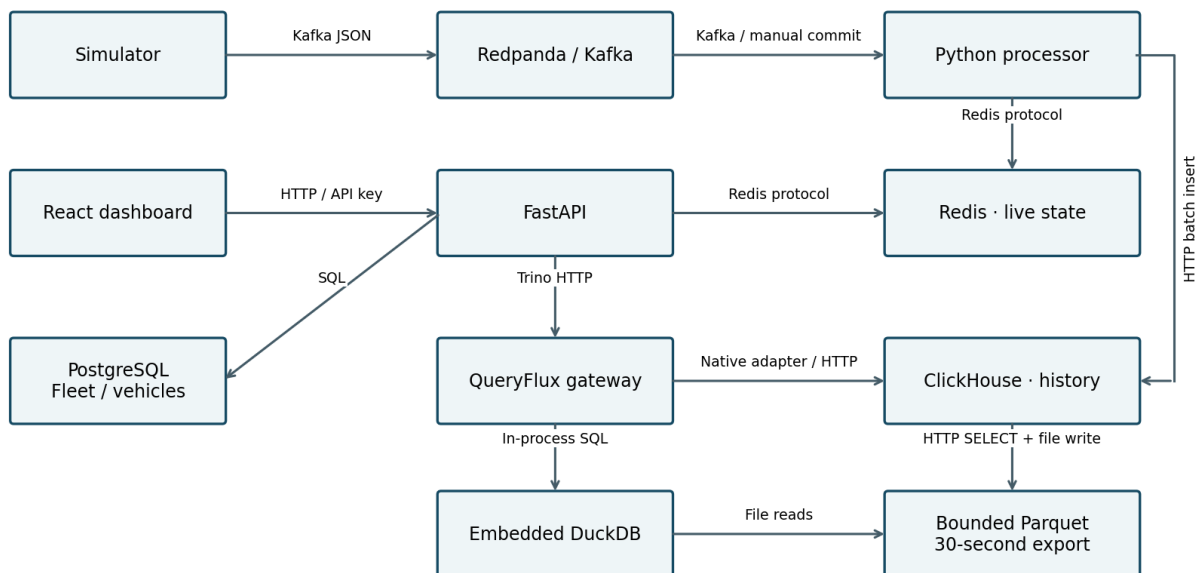
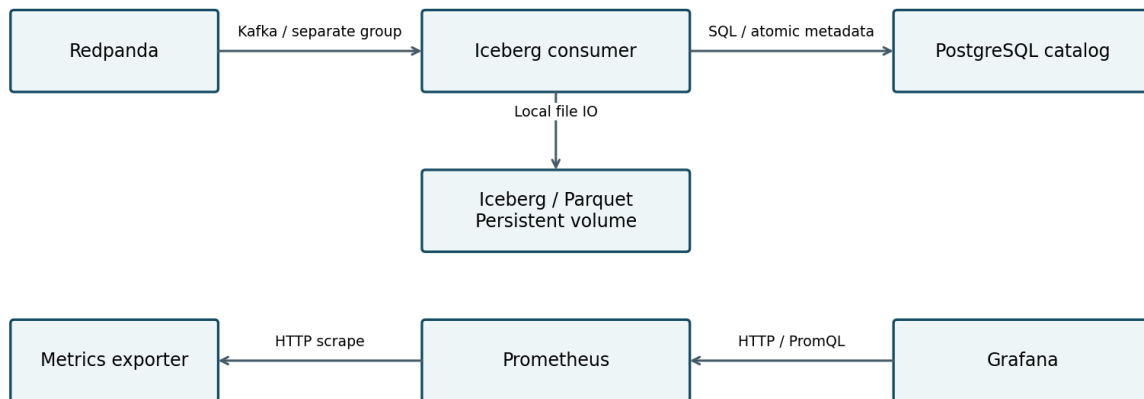


Figure 3. Core service/data-store view. Arrows identify protocols or file access; direct ClickHouse mode is an explicit alternative API configuration.

Optional profiles · archive and monitoring



Exporter reads authenticated API metrics; Prometheus also scrapes native QueryFlux metrics.

Figure 4. Optional archive and monitoring profiles. Archive progress is independent of live processing.

Hop	Mechanism / configured bound	Latency evidence
Simulator → Kafka	vehicle_id key; six partitions; producer linger 20 ms	Configured batching, not measured hop latency
Kafka → processor	Manual commits; up to 500 messages per batch	No per-hop percentile distribution
Processor → history/live	ClickHouse insert then Redis transaction, then offset commit	Latest-batch latency gauge; not a percentile
Stores → operator	API request; independent UI polling every 3 seconds	<2-second dashboard target not established
History → DuckDB sample	Refresh every 30 s; reject snapshot older than 120 s	Bounded sample, not instantaneous full history

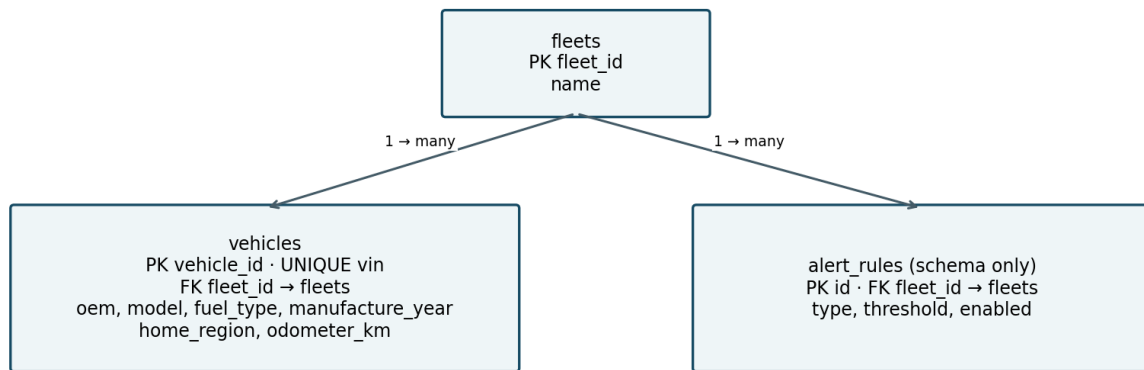
5.2 Technology Stack & Justification

Layer	Choice	Why / alternatives and trade-offs
Ingestion / messaging	Redpanda, Kafka protocol	Partition ordering and replay instead of direct synchronous writes; one broker reduces demo overhead but lacks HA.
Processing	Python, Pydantic, bounded batches	Small testable processor instead of introducing Flink/Spark; single owner limits scale.
Live / metadata / history	Redis; PostgreSQL; ClickHouse	Point reads, relational constraints and columnar scans separated instead of forcing every workload into one store.
Analytics / archive	Existing QueryFlux; DuckDB; optional Pylceberg	Explicit engine routes and bounded sample; archive independently preserves raw Kafka identity.
Backend / frontend	FastAPI; React/Vite	Typed request validation, OpenAPI and independently refreshing panels; polling chosen over a WebSocket service.
ML / AI	Deterministic threshold rules	Explainable baseline; no trained model or LLM in runtime.

Layer	Choice	Why / alternatives and trade-offs
Infrastructure / CI / observability	Docker Compose; GitHub Actions; optional Prometheus/Grafana	Reproducible local services; Kubernetes and multi-cloud deployment deferred.

5.3 Data Architecture

Metadata relationships · PostgreSQL



Runtime rules use environment settings; OEM/model catalog normalization is deferred.

Figure 5. Implemented metadata ER view (infra/postgres.sql). Full OEM/model catalog normalization is deferred.

Normalization: fleets and vehicles are separated with foreign keys and a unique VIN constraint. OEM/model/fuel attributes deliberately repeat per vehicle; full catalog 3NF is not claimed. ClickHouse repeats fleet/vehicle attributes to avoid metadata joins in telemetry scans. alert_rules is a schema placeholder; detector rules currently come from environment settings.

Store / tier	Data and retention	Consistency / partition trade-off
Redpanda buffer	6 partitions, 1 replica; 24 h or 256 MiB per partition, whichever limit applies	Single broker; no multi-node CP/AP guarantee established.
Redis hot state	Latest state/alerts: 1 h; dedup: 24 h; 256 MiB noeviction	Processing fails closed on sink/lease failure; favors stopping over uncertain progress.
PostgreSQL metadata/catalog	Fleets, vehicles; optional Iceberg SQL catalog	Single transactional authority; unavailable if this node fails.
ClickHouse warm history	Monthly partitions; (vehicle_id, timestamp, event_id) ordering; 30-day TTL	ReplacingMergeTree + FINAL dedup views; no atomic transaction with Redis.
DuckDB sample	≤120 min, first 100 vehicle IDs, ≤20,000 rows; 30 s refresh; max age 120 s	Derived snapshot; stale sample returns 503 rather than a successful stale answer.
Iceberg cold archive	Daily received_at partitions; raw bytes and Kafka coordinates; no automatic TTL	Data and next-offset properties commit atomically; one advisory-lock owner. No cross-store transaction.

CAP labels depend on deployment and failure conditions. This single-node prototype does not demonstrate partition-tolerant multi-node CP or AP behavior. It explicitly stops live processing when sinks cannot acknowledge and rejects stale DuckDB samples, sacrificing availability in those tested paths.

PACELC: without a partition, direct Redis reads favor low live-read latency while ClickHouse FINAL applies deduplication work at query time; DuckDB trades freshness/completeness for a bounded local sample. During failures, required sink acknowledgements and sample freshness checks take priority over continued successful responses. This explains local consistency/latency choices; replication latency and multi-node partition behavior have not been measured. Storage cost is not priced: no cloud region, service size or verified pricing basis is selected, so the byte estimates below are not a monetary cost estimate.

Capacity estimate uses an assumed decimal 1 KB/event before compression, replicas, indexes and metadata. Required rate = active vehicles × readings/second. These are planning calculations, not measured disk usage:

Event rate	Daily volume	Yearly volume (365 days)	Validation
1K/sec	86.4 GB	31.536 TB	Short generation target stable
10K/sec	864 GB	315.36 TB	Short run unstable
100K/sec	8.64 TB	3.1536 PB	Not attempted

Partition keys: Kafka uses vehicle_id; ClickHouse uses monthly event-time partitions and a vehicle/time/event ordering key; Iceberg uses daily received_at partitions, preserving event time separately. Archive coverage begins at retained Kafka offsets. It cannot recover already expired records or historical ClickHouse-only data.

Query	Before (ms)	After (ms)	Change / evidence
Vehicle history, latest 100 rows	23.693	2.402	substring(vehicle_id, 2) → exact vehicle_id equality; 515,271 → 4,096 rows read.
Fleet idling aggregation	Not measured	Not measured	Evidence gap: save real before/after plans, equal results and timings.
Event activity aggregation	Not measured	Not measured	Evidence gap: save real before/after plans, equal results and timings.

One of the requested three comparisons exists. Recorded evidence is ClickHouse EXPLAIN indexes = 1 plus five actual execution statistics per variant, not an EXPLAIN ANALYZE report. The equality predicate changes primary-key pruning from 474/474 to 4/474 granules; results match. Full plans and runs: docs/sql-optimization.md and artifacts/sql-optimization.json. The submitting team should confirm acceptance of this engine-specific equivalent and add two genuine comparisons.

5.4 Deployment View

One development host / Docker Compose network
 Browser → 127.0.0.1:3000 (frontend proxy) → backend:8000
 One broker + one processor + one Redis + one PostgreSQL + one ClickHouse
 QueryFlux + DuckDB bootstrap / snapshot worker (root demo)
 Optional: monitoring profile; iceberg profile
 Persistent Docker volumes: databases, snapshots and archive files
 Ignored .env: generated local credentials / configuration

Root docker compose up --build -d starts the configured two-engine demo and simulator after setup. Published service ports bind to loopback. Services use one replica, health checks and restart policies; no Kubernetes namespace, autoscaler, load balancer or cloud-managed secret store is configured. The same images/Compose definitions can be moved to another suitable host, but no second-cloud deployment has been tested. Availability remains tied to one machine and its volumes.

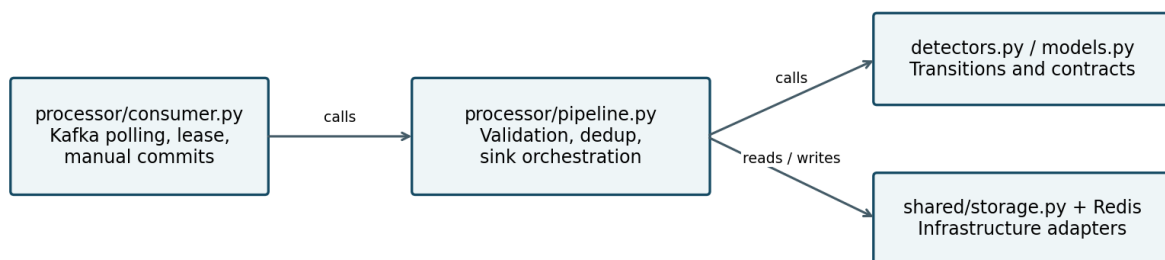
PDF requirements, pages 6 and 9–10: a deployment on at least one cloud, portability to another without code changes, Helm charts or Kubernetes manifests, and Terraform for one cloud. These artifacts and a verified cloud deployment are absent. Dockerfiles and Compose alone do not complete that DevOps pack.

6. Low-Level Design

6.1 Layering & Separation of Concerns

The implementation is a pragmatic layered service design. Detectors are independently testable, but SQL templates remain in API handlers and the processor knows its sink adapters; strict hexagonal dependency inversion is not claimed.

Processor layers · pragmatic dependency direction



Detectors do not import API/database clients. Pipeline deliberately knows its sink adapters.

Figure 6. Allowed call direction in the processor and the explicit infrastructure coupling.

Layer	Responsibility / code	Boundary / current exception
Presentation / API	React UI; FastAPI validation, API key, DTOs	UI contains no database clients; backend/main.py currently also contains SQL templates.
Application / service	Processor sequencing; analytics route selection and admission	Orchestrates adapters; depends on concrete Redis/ClickHouse clients in this prototype.
Domain	shared/models.py; processor/detectors.py	Typed event constraints and incident transitions; no HTTP/DB client calls.
Infrastructure	shared/storage.py; Kafka producer/consumer; archive/catalog clients	Encapsulates external IO; no claim of full vendor-type isolation.

```

simulator/  cli.py, generator.py, producer.py, vehicle_factory.py
processor/  consumer.py, pipeline.py, detectors.py
shared/    models.py, config.py, storage.py
backend/   main.py, analytics.py, monitoring.py, queryflux/
archive/   cli.py, consumer.py, table.py, config.py
frontend/  src/, scripts/, package.json
infra/     docker-compose.yml, postgres.sql, clickhouse.sql, monitoring/
scripts/   setup_env.py, benchmark.py, duckdb_snapshot.py, verify_dual_engine.py
tests/     unit/, integration/
docs/      architecture.md, verification.md, adr/, iceberg.md
artifacts/ saved JSON reports, logs and product screenshots

```

6.2 Design Principles Applied

SOLID: detector transition logic has one focused responsibility; storage and QueryFlux clients have narrow interfaces, and tests inject doubles into Processor/create_app. Dependency inversion is partial because production orchestration still constructs concrete clients. Settings, canonical models and shared adapters reduce repeated logic (DRY); explicit thresholds and bounded batches keep the design understandable (KISS).

12-factor practices: environment configuration, separate worker/API processes, pinned dependency locks and container logs. State is external to application processes; graceful producer flush and manual offset commits support disposability. Local persistent volumes and bootstrap jobs are development deployment choices, not full cloud-native qualification.

Idempotency: deterministic event/alert keys, in-batch sets, Redis SET NX with TTL and ClickHouse FINAL views. Fail-fast: invalid settings, lost lease, sink failure, untrusted gateway nextUri and stale samples return explicit failures. Least privilege is partial: API-key checks, loopback ports and unprivileged application containers exist; tenant roles and separate read-only analytical credentials remain work.

6.3 Design Patterns Used

Pattern	Problem solved	Code
Adapter	Wrap database and analytical gateway operations	shared/storage.py; backend/queryflux/client.py
CQRS-style workload separation	Use distinct live/metadata/analytical read paths	backend/main.py; processor/pipeline.py
Explicit strategy / routing	Select direct vs QueryFlux mode and sample namespace	backend/analytics.py; infra/queryflux.yaml
Bulkhead / bounded admission	Limit analytics to four running and sixteen waiting requests	backend/analytics.py
Idempotent consumer / checkpoint	Replay protection; atomic archive offsets with data	processor/pipeline.py; archive/consumer.py

No saga, transactional outbox, distributed exactly-once transaction or dedicated circuit breaker is implemented. Retry/restart behavior is bounded by producer/client settings and worker failure semantics.

6.4 Interfaces, Contracts & Runtime Flows

Contract	Implemented behavior
OpenAPI / version	http://localhost:8000/openapi.json and /docs; endpoints under /api/v1

Contract	Implemented behavior
Authentication	X-API-Key; constant-time comparison; no OIDC/RBAC
Pagination / bounds	Vehicles: limit 1–100 and offset ≥ 0 ; history: 1–30 days, limit $\leq 1,000$; validated vehicle/fleet IDs
Error envelope	<code>{"error":{"code":503,"message":"A required data service is unavailable"}};</code> validation errors use 422 with field locations
Admission / rate limits	Four concurrent analytical requests, sixteen waiting; bounded query templates; no general per-user request rate limiter
Event topic / key	valeosense.telemetry.v1; key vehicle_id; JSON validated by shared/models.py
Schema / evolution	schema_version=1; UTC timestamp; UUID event_id; extra fields rejected; no schema registry or migration pipeline

Canonical events include vehicle/VIN/fleet identity, event time, sequence, latitude/longitude, speed/heading, engine state, fuel or SOC, odometer, engine temperature, DTC list, event_type, source_oem and schema_version. Exactly one of fuel_pct/soc_pct is populated. Unknown formats are rejected by the live processor; the optional archive preserves invalid raw payloads with an error classification.

Runtime sequence · successful batch

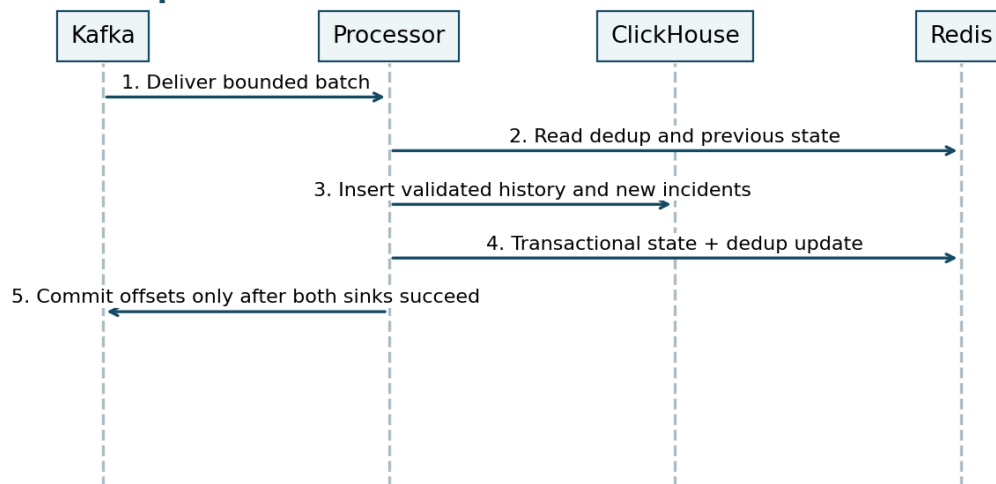


Figure 7. Successful processing. Domain validation and detector transitions occur before inserts.

Failure sequence - Redis write fails after history insert

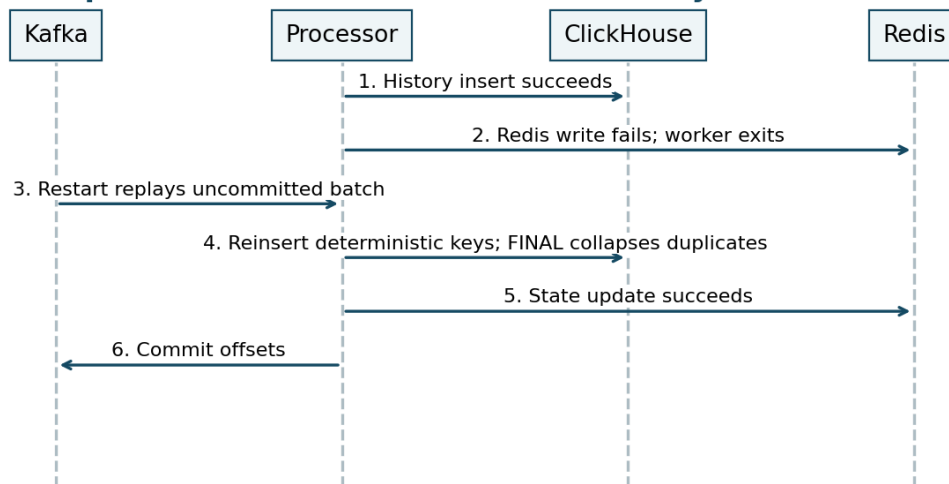


Figure 8. Tested recovery pattern. Practical replay protection does not make Redis and ClickHouse one atomic transaction.

6.5 Algorithms & Data Structures

Detectors use previous per-vehicle state and episode state. Idle means engine on and speed ≤ 0.5 km/h; continuity resets after a gap over ten seconds. The default idle alert requires more than 20 seconds. Acceleration = $\Delta \text{speed} / 3.6 / \Delta \text{time}$; harsh defaults are below -3 or above $+3$ m/s², and speeding is above 100 km/h. Faults inspect DTC contents. Episodes suppress repeated new incidents while active alerts update.

```

for payload in bounded_batch:           # O(B)
    validate canonical schema and event time
    skip event IDs already in in_batch_set # expected O(1)
read dedup markers and previous state from Redis
for event in valid_events:
    if retained dedup marker exists: continue
    if event is late: append history only
    else: compute transition; append history + new incidents
insert ClickHouse history / incidents
commit Redis state + dedup markers      # sorted sets O(log V)
commit Kafka offsets only after both sinks acknowledge
    
```

Time: expected $O(B)$ validation/hash/scalar transition work for batch size B , plus $O(B \log V)$ sorted-set updates for V active vehicles and network/storage costs. Space: $O(B)$ batch buffers, $O(V)$ live state and $O(\text{event rate} \times \text{dedup TTL})$ retained markers. DTC lists are capped at sixteen. The benchmark exercised 1K and 10K active vehicles with the recorded generation rates; no isolated per-function runtime or 100K/sec processing claim is made.

7. Non-Functional Requirements & Performance Benchmarks

NFR	Template target	Achieved / gap	How measured
Ingest throughput	Sustain 100K+ events/sec	Not achieved. Stable short 1K target; 10K unstable.	15 s real pipeline stages; benchmark_results.json
Burst tolerance	3× burst for 5 minutes without loss	Not tested at 300K/sec; simulator burst behavior alone is insufficient	PDF page 9; no qualifying load report

NFR	Template target	Achieved / gap	How measured
Horizontal scalability	Add service/broker nodes without code changes	Partial service boundaries; processor and archive enforce one owner	No scale-out demonstration
End-to-end latency	<2 s dashboard; <5 s critical alert	Not established. UI polls every 3 s.	No event-to-visible percentile benchmark
API latency	p95 <200 ms; p99 <500 ms	Not measured	Single query observations and means are not percentiles
Resilience	Recover after broker/pod failure	Tested duplicate/sink replay and stale-sample recovery; broker crash campaign absent	Integration checks and docs/verification.md
Availability	99.9%; no single point of failure	Not provided: one host, one broker and one state owner	Architecture/configuration inspection

Load setup: scripts/benchmark.py runs the real generator → Kafka → processor → Redis + ClickHouse path. The saved host had eight logical CPUs and 7,988,129,792 bytes RAM (about 8 GB). Each requested stage lasted 15 seconds, optional monitoring was enabled, and no concurrent demo simulator was allowed. Passing requires $\geq 90\%$ generation target, exact published/consumed counts, the same processor instance, <5 seconds drain and no processor errors.

Target	Generated/sec	Consumed/sec incl. startup/drain	Drain	Status
1,000/sec	999.87	923.99	0.50 s	Stable short stage
10,000/sec	9,997.86	3,762.36	24.53 s	Unstable
25K / 50K / 100K	Not attempted	Not attempted	N/A	Stopped after saturation

Saved real pipeline benchmark · 27 September 2026 · 15-second stages

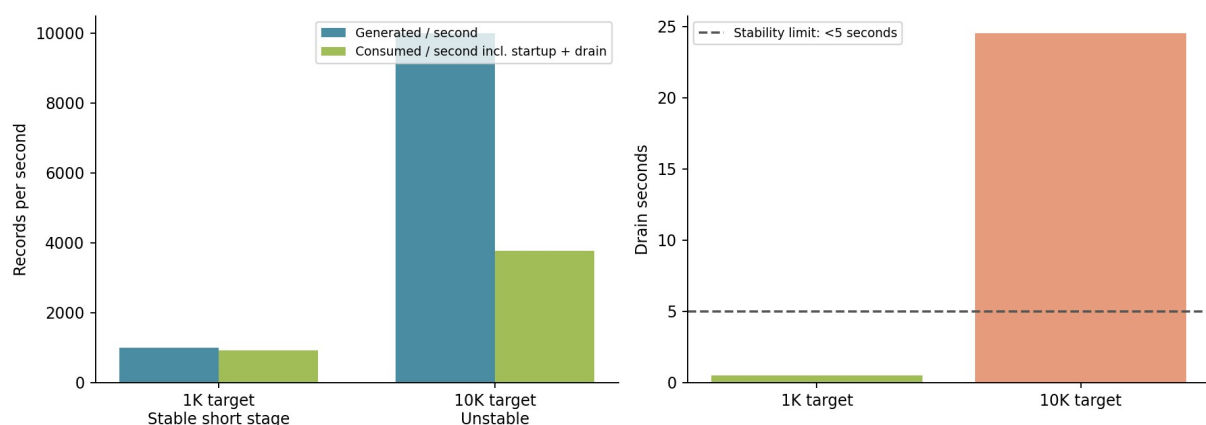


Figure 9. Chart generated from benchmark_results.json; measured 27 September 2026. Intentional duplicate/malformed scenarios make generated, published, accepted and consumed counts distinct.

At the 1K stage, 15,000 readings were generated, 15,270 records published and consumed, 14,925 accepted, 270 duplicates ignored and 75 schema errors rejected. At 10K, 152,595 records were published and consumed after the long drain. These short stages are not a soak test. API p95/p99, a full end-to-end latency distribution and sustained 100K/sec remain evidence gaps.

8. Security & Compliance

Threat / STRIDE category	Implemented controls	Remaining limitation
Forged or modified telemetry (S/T)	Loopback broker access, canonical schema, numeric bounds and event-time validation	No producer identity, payload signatures or mTLS
Unauthenticated or cross-fleet access (S//E)	API key, constant-time comparison and bounded validated identifiers	Single demo operator; no tenant isolation, OIDC or RBAC
SQL injection (T/E)	Parameterized metadata SQL, fixed analytical templates, no arbitrary SQL endpoint	Separate read-only analytical credentials not implemented
Secret / location disclosure (I)	Ignored locally generated credentials, loopback ports, synthetic locations	No service TLS, managed secrets or verified at-rest encryption
Exhaustion / replay / provenance (D/R)	Bounded queues/admission, dedup, event IDs, offsets and logs	Finite dedup TTL; no immutable audit log or broad rate limiter

Authentication and authorization: protected API routes use a static local key; the frontend stores the key for the browser session. CORS allows configured origins and is not authentication. /health and /docs expose health/schema without a key. QueryFlux uses configured credentials; Grafana requires login. Device identity, tenant isolation, fleet-level authorization and service-to-service authentication are incomplete.

Privacy: vehicle IDs, VINs, telemetry and locations are synthetic. No real driver records are intentionally used. Live state expires after one hour, ClickHouse history after 30 days and Kafka by configured time/size limits. The optional Iceberg archive has no automatic expiry; deletion/retention maintenance and a complete erasure workflow are not implemented. No GDPR/DPDP compliance assessment or certification is claimed.

AI safety: not applicable to runtime because there is no LLM or autonomous agent in the processing path. Security scan coverage and a complete transitive SBOM remain missing evidence. This section reports implementation state, not a legal or security certification. Source: docs/threat-model.md.

The PDF page 9 goes further than local API-key protection: it requires OAuth2/OIDC + JWT, RBAC/tenant isolation, device mTLS, TLS 1.3, AES-256 at rest, secrets in a vault, OWASP Top 10/API Top 10 controls, audit logs for each data access, location masking and verified erasure. These are missing or unverified. Existing operational logs are not a per-access audit trail; synthetic data avoids real-person exposure but does not implement the requested privacy workflows.

9. Test Strategy

Test type	Tools	Count / saved result	Coverage / limitations	In CI?
Unit / API / component	pytest, doubles, temporary Iceberg tables	106 passed; 3 opt-in integrations skipped (30 Sep)	Includes 14 archive component cases; line/branch coverage not measured	Yes; hosted run not checked here
Integration / contract	Real Kafka, Redis, PostgreSQL, ClickHouse, QueryFlux	2 core/dual-engine cases passed (28 Sep UTC); 1 archive case passed on host and in container (30 Sep)	Separate runs; do not add repeated host/container execution as distinct tests	No; opt-in

Test type	Tools	Count / saved result	Coverage / limitations	In CI?
Acceptance / browser	Node request tests, Playwright, Chromium smoke	3 request + 4 Playwright tests passed; browser smoke passed	Saved two-engine verification; no formal BDD feature suite	No
Performance / load / soak	Custom Python benchmark; ClickHouse plans	2 load stages; 1 optimization comparison	1K stable, 10K unstable; no soak or p95/p99 report	No
Security SAST/DAST/dependencies/images	No saved scanner campaign	Not run / no report	Ruff is lint, not a complete security scan	No
Compliance / chaos	Replay and fault-oriented tests	Targeted paths verified; no separate campaign count	No broker-cluster chaos, compliance certification or HA qualification	Partial unit cases only

Edge cases covered by code/tests include repeated events, late timestamps, non-finite/out-of-bounds values, invalid DTCs, mutually exclusive fuel/SOC, long continuity gaps, producer delivery failure, sink failure, cancelled analytical waiters, untrusted pagination origins, stale/missing metrics, archive tombstones, uncertain catalog responses and failed Kafka acknowledgements. Unknown OEM formats are schema failures, not an implemented OEM adaptation layer.

Illustrative input/output contract: for consecutive valid engine-on readings at 0 km/h, the first interval beyond the configured 20-second idle threshold creates an IDLING_ALERT; repeating the same event ID does not create another incident while dedup state is retained. This is a contract example, not an additional measured experiment.

Report locations: docs/verification.md, artifacts/browser-smoke.json, artifacts/dual-engine-lineage.json, artifacts/monitoring-smoke.json, benchmark_results.json and artifacts/sql-optimization.json. Some archive results are recorded in the verification narrative rather than a committed standalone raw test log. The current CI workflow runs Ruff, pytest and the frontend build; external integration/browser/security checks require additional jobs or explicit disclosure.

PDF pages 9–10 specifically require $\geq 80\%$ coverage on core services/algorithms, BDD acceptance scenarios, real-service integration/contract evidence, a soak test, SAST/DAST/dependency/image scans, audit/erasure tests, broker/pod kill recovery and a CI pipeline link. No coverage report, formal BDD suite, consumer-driven contract suite, soak/security campaign, erasure test or broker-kill recovery report is present. Candidate pipeline URL:

<https://github.com/Kaileshwar16/VeloSense/actions> (inferred from origin; successful final run and reviewer access must be checked).

10. Observability

The API exposes request/error counts, workload counts, process heartbeat, generator/consumer rate, committed lag and analytical route information. Optional exporter → Prometheus → Grafana collects operational history; Prometheus separately scrapes native QueryFlux engine counters and duration metrics. API route labels identify intended workloads; native counters provide independent execution evidence.

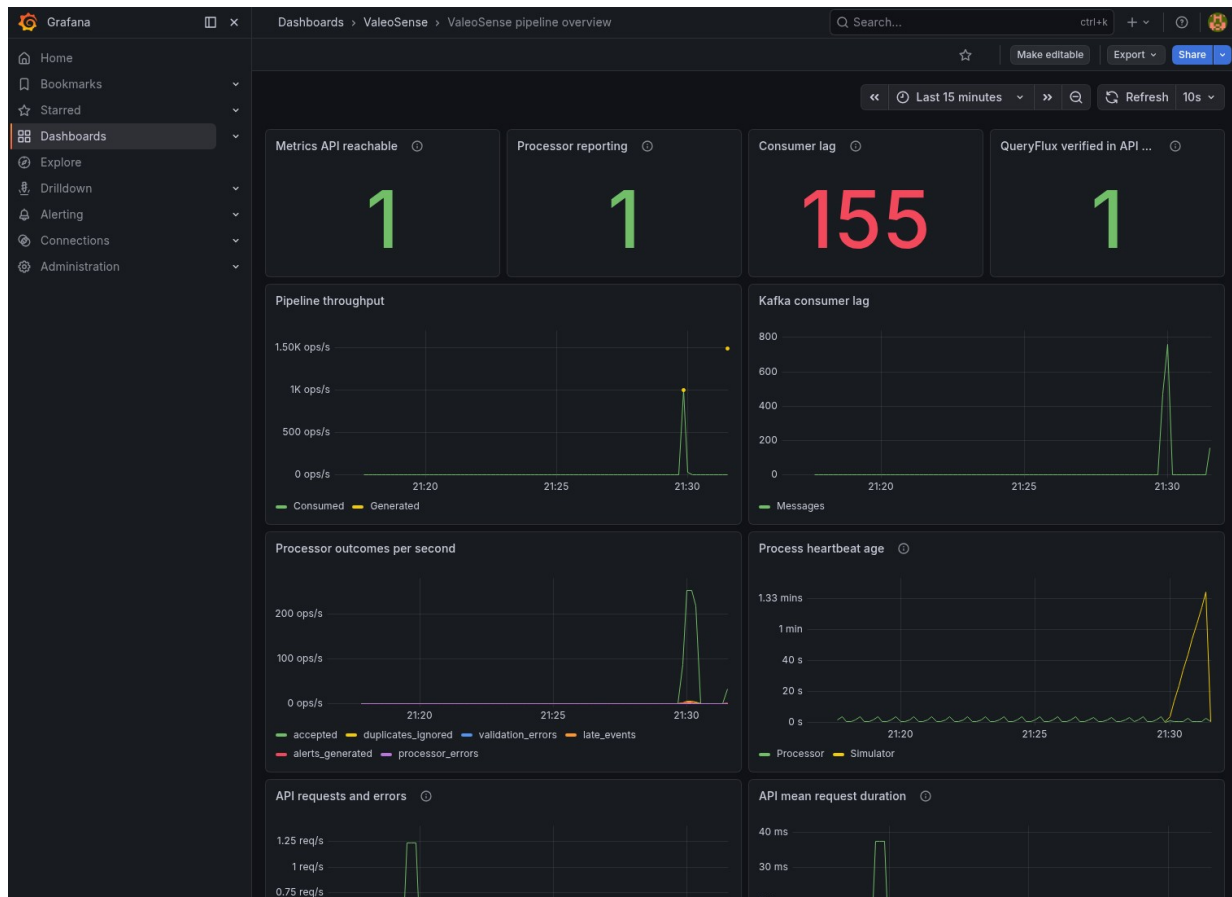


Figure 10. Saved Grafana screenshot showing throughput, lag, outcomes and heartbeat. This earlier capture is evidence of rendering; the later verification record reports 18 panels. No current health claim is inferred from the screenshot.

Troubleshooting a latency spike: (1) check scrape health, processor heartbeat and fresh vehicle timestamps; (2) compare producer and consumer rates with committed lag; (3) inspect processor logs and Redis memory, especially the 256 MiB noeviction limit; (4) compare ClickHouse/QueryFlux errors and query-duration observations; (5) check snapshot age if only DuckDB requests fail; (6) rerun the bounded verification after the fault is resolved. A healthy /health response alone does not prove fresh telemetry.

Prometheus scrapes every ten seconds and has bounded local retention. Counters are process-local. API mean duration and latest-batch event latency are not p95/p99. Distributed traces, a tracing UI and outbound alert notifications are not implemented; the missing end-to-end latency percentiles remain a requirement gap.

PDF page 7 also calls for centralized logs, metrics and traces. Metrics are implemented in the optional profile; logs remain local/container logs and there is no central log backend or distributed tracing. This expectation is only partially met.

11. AI / ML Component (if used)

Not applicable: the runtime uses deterministic rules and per-vehicle episode state, with no trained ML model, vector store, retrieval pipeline or LLM agent. Explicit thresholds are sufficient for the prototype's observable incident definitions; predictive maintenance accuracy is not claimed.

Data/features: synthetic physical telemetry, prior event state, elapsed time and diagnostic codes. Model evaluation, leakage checks, hallucination controls, per-request LLM costs and tool-access policies do not apply to this implementation. Codex assisted engineering and documentation; it is not part of the production event path. Any future predictive model would require labelled evaluation data and comparison against these rules before claiming benefit.

Scope ambiguity to resolve: PDF page 6 says vector storage is needed “where useful”; pages 7–8 nevertheless list an ML/vector layer, an agent with guardrails/audit and combined vector storage among technical expectations. No such component is implemented. TEAM ACTION — confirm whether the chosen deterministic, non-AI problem scope is accepted; if not, this is an additional implementation requirement, not a completed feature. Graph/DP algorithms and specific opportunity examples should likewise be selected only where relevant to the chosen problem.

12. Architecture Decisions, Risks & Future Enhancements

ADR / context	Options → decision	Consequences
001: ordered replayable ingestion	Direct writes / Kafka / Redpanda → one Redpanda broker, six partitions	Simple Kafka-compatible demo; no replication-based HA.
002: live state and continuity	Relational-only / process memory / Redis → Redis with TTL and lease	Restartable hot state; finite memory; on failure stop rather than commit uncertain work.
003: metadata vs event history	One general store / specialized stores → PostgreSQL + ClickHouse	Constraints plus analytical scans; cross-sink transaction absent.
004: analytical route	Direct ClickHouse / external gateway → QueryFlux with native history and bounded DuckDB sample	Externally attributed dependency; explicit rules, no cost-based routing/failover.
005: bounded demo scope	Large orchestrated stack / minimal local services → Compose and optional profiles	Local Iceberg/monitoring possible; cloud, maintenance and HA qualification deferred.

CAP-related choice: ADR-002 deliberately stops ingestion when Redis cannot maintain trustworthy state/ownership; ADR-004 rejects stale samples. This is an availability trade-off in known failure paths, not proof of cluster-wide CP behavior. Full decision records: docs/adr/001-streaming.md through 005-scope-and-retention.md.

Known risks: one state owner; finite broker retention; Redis dedup memory growth; no Redis/ClickHouse atomic transaction; malformed live records lack a dedicated dead-letter queue; no user/tenant authorization; short benchmark evidence; incomplete query comparisons/security scans; local archive growth without compaction/expiry. Archive verification explicitly excluded a pre-existing restarting live processor, so that record alone cannot establish current dashboard health.

Next three steps toward a pilot: (1) strengthen recovery with partition-owned workers, tested checkpoints, capacity/latency soak tests and failure drills; (2) implement authenticated tenant boundaries, producer identity, encrypted transport and measured security controls; (3) qualify durable archive storage and backup/restore, compaction and retention, then run a small fleet pilot with validated business metrics.

13. Demo Video (5 Minutes Maximum)

TEAM ACTION — record and upload the final working-product demo, no longer than 5:00, at 1080p with clear audio; captions are recommended. Supply the accessible URL here and on the cover, then replace feature-table Pending cells with actual timestamps. The following times are a proposed storyboard, not claims about an existing video.

Time	Segment	What to show
0:00–0:30	Problem	Fleet manager need; 100,000 registered synthetic vehicles; distinguish active subset.
0:30–1:00	Solution	Architecture and separate live/history read paths.
1:00–3:00	Live demo	Advancing telemetry, an actual detected incident, vehicle history, bounded DuckDB sample and native route evidence.
3:00–4:15	Under the hood	Recorded replay integration result, benchmark limits and one real SQL optimization; optional monitoring.
4:15–5:00	Impact / next steps / team	Measured engineering results, unvalidated business impact, next steps and team.

Video link: TEAM ACTION — insert final YouTube/Vimeo/Drive URL with reviewer access.

Detailed click-by-click preparation and narration are already in docs/demo-recording-guide.md (target 4:50). Rehearse on fresh telemetry and verify both native engines before recording. Keep API keys and local credentials off-screen. An integration replay clip proves its stated duplicate/sink-replay path; it does not prove broker-cluster failover or exactly-once cross-store transactions.

14. Repository Checklist

Requirement	Current status	Remaining action
README and startup	README, environment example, diagrams, run guide and commands present	Fresh-clone rehearsal; confirm reviewer can reproduce setup.
One-command configured run	docker compose up --build -d starts root demo after setup	Initial setup downloads dependencies/images; running stack not refreshed during document edit.
Service / docs / infra / tests structure	Present; source revision c5549e8	Keep final code and submission artifacts consistent.
CI build, lint, all tests, security	Ruff, pytest and frontend build configured	Integration/browser/security suites not included; add or explicitly disclose.
Secrets / .env.example / contributions	.env ignored; example present; generated secrets local	Review final tracked files, attribution and authentic member contributions; no history scan claimed.
Repository URL / access	origin: https://github.com/Kaileshwar16/VeloSense	Verify final remote code, permissions and reproducibility.
Final tag v1.0-submission	Absent from local tag list on 1 Oct	Create on reviewed final commit and push when ready.

Requirement	Current status	Remaining action
Final PDF / video	Document draft and PDF preview prepared; no video supplied	Finalize highlighted fields, refresh PDF and submit the actual package.

TEAM ACTION — finalize team metadata, video and timestamps, review the technical gaps, verify repository access and the final tag, then export the completed Word document to PDF. This edit did not stage files, commit, push, upload a video or submit to the organizer.

15. Conclusion

ValeoSense demonstrates a complete synthetic event-to-dashboard path with bounded processing, incident detection, independent live and analytical stores, explicit two-engine lineage and an optional raw archive. The strongest evidence is reproducible local behavior: deterministic registry checks, real service integration, replay tests and measured query pruning.

Key learnings are that fleet registry size differs from streaming capacity, deduplicated views do not create cross-store exactly-once transactions, and configured routes need native execution evidence. Timestamp/late-event handling, stale sample failure behavior and processor saturation were tested or documented. Moving to a pilot requires stronger recovery, tenant security, operational maintenance and sustained performance evidence; customer impact remains to be validated.

16. Declarations

External component attribution: QueryFlux is an existing external project (<https://github.com/lakeops-org/queryflux>), not software created for this submission. The project integrates it through configuration and clients. The following are declared licenses recorded in docs/open-source.md from the 27–30 September checks; this is not a full transitive SBOM or a new legal audit.

Component	Declared license / qualification
QueryFlux; ClickHouse	Apache-2.0; existing external components.
DuckDB / libduckdb-sys	MIT; local libduckdb-sys 1.10501.0 LICENSE and package metadata inspected for this document.
Redpanda v25.1.9	BSL-1.1 source-available; not classified as permissive open source.
Redis server 7.4	RSALv2 or SSPLv1; distinct from redis-py MIT client.
PostgreSQL 17	PostgreSQL license.
Pylceberg / pyiceberg-core / PyArrow	Apache-2.0; optional archive dependencies.
SQLAlchemy; FastAPI; Pydantic; redis-py; React; Vite; pytest; Ruff	MIT (bundled/transitive components may carry separate notices).
Uvicorn; HTTPX; python-dotenv; psutil	BSD-3-Clause.
confluent-kafka; Playwright; Prometheus	Apache-2.0; bundled native packages retain their notices.
Psycopg	LGPL-3.0-only; binary distribution has additional notices.
Grafana OSS	AGPL-3.0.

AI tools used: Codex assisted coding, debugging, tests and documentation, including preparation of this solution document. No LLM API or ML model runs in the vehicle data path. Claims are grounded in source and saved outputs rather than generated benchmark or customer evidence.

Data declaration: the vehicle registry, telemetry, VINs and vehicle locations used by the demo are synthetic; no real customer/driver records are intentionally included. Screenshots are saved captures of the working local product. Estimates and unsupported production targets are labelled. TEAM ACTION — confirm the final submission contains only synthetic/public authorized data and confirm member/third-party attribution. The design, problem framing and authored code must be the team's own work, with external components and AI assistance declared.

Affiliation: this is an independent academic exercise. Motorq is used only as an industry reference; no affiliation with Motorq is claimed (PDF page 10). Code freeze: only commits before the organizer's deadline are reviewed. The supplied PDF does not state the deadline timestamp or submission portal; confirm both with the organizer before finalizing v1.0-submission.

17. Appendix (if any)

Evidence index

Evidence	Repository path / date
Implementation baseline	c5549e8; document prepared 01 Oct 2026
Registry identity	artifacts/registry-verification.json — 27 Sep
Real pipeline benchmark	benchmark_results.json; docs/benchmark.md — 27 Sep
Query optimization	artifacts/sql-optimization.json; docs/sql-optimization.md — 27 Sep
Engine lineage	artifacts/dual-engine-lineage.json; dual-engine-stream.json — 28 Sep UTC
Browser / operational evidence	artifacts/browser-smoke.json; monitoring-smoke.json; grafana-browser-smoke.json; PNG captures
Latest archive verification	docs/verification.md, Iceberg section — 30 Sep; docs/iceberg.md
Contracts / schemas	shared/models.py; infra/postgres.sql; infra/clickhouse.sql; archive/table.py
Threats / decisions / attribution	docs/threat-model.md; docs/adr/; docs/open-source.md
Run and demo guides	RUN_PROJECT.md; docs/demo-recording-guide.md; docs/failure-demo.md

Reproduction commands

Run from the repository root after prerequisites and local configuration in RUN_PROJECT.md. These are reproduction instructions; they were not executed as fresh runtime checks during document preparation.

```

docker compose up --build -d
make lint
make test
make frontend-build
make verify-dual-engine
make integration-queryflux
make browser-test
make monitoring
EXPECT_QUERYFLUX=1 make monitoring-test
make iceberg
make iceberg-status
make iceberg-integration

```

Benchmark and SQL evidence should be run only in their documented controlled setup: stop competing demo traffic, require known-zero lag and preserve existing evidence before refreshing it. Use make benchmark and make sql-evidence; record measurement date, commands and raw outputs. Do not substitute a screenshot rate or one query latency for a sustained capacity or percentile result.

Outstanding submission requirements

TEAM ACTION — complete team name, member names/roles/emails, actual submission date, video URL and feature timestamps. Verify remote access, a passing final CI run and the deadline/tag; review and export the final PDF. Resolve the missing cloud/IaC, scale/burst/latency/HA, security/privacy, coverage/BDD/chaos/CI evidence and two SQL comparisons, or obtain an organizer-approved scope exception. Disclosure makes the submission honest but does not fulfill an unmet mandatory requirement. The companion docs/submission-remaining-requirements.md maps these gaps to PDF pages.

PDF requirement cross-check

PDF reference	Already present	Missing or partial
§6, p6: minimum bar	Own simulator, 100K registry, real-time pipeline, historical analytics, relational + Redis, UI, Docker	Only active subsets tested; secure enterprise controls incomplete; cloud deployment unverified
§7–8, p7: technical/data	Schema validation, replay/backpressure, specialized stores, one SQL comparison	Rate limiting, strict catalog 3NF, billions-row proof, central logs/traces, cloud/IaC, AI/vector scope clarification
§9–10, p8: algorithms/system design	Complexity write-up, CAP/PACELC discussion, hot/warm/cold design and 5 ADRs	Monetary storage cost estimate; demonstrated replication/scale-out/partition behavior
§11, p9: NFRs	Bounded prototype and short capacity evidence	100K sustained + 3× burst/5min; required latency; HA; enterprise identity/encryption/privacy
§12, p9: testing	Unit/component tests, real integration, browser checks, short load evidence	80% coverage, BDD/contracts, soak, scanner reports, audit/erasure/chaos, all suites in CI
§13–14, p9–10: deliverables/rules	Source, README, Compose, diagrams, ADRs, STRIDE, algorithms and declarations	Team/video/PDF finalization; cloud + Helm/K8s/Terraform; final CI link, tag/deadline and reviewer access